

Bit Manipulation DSA Handbook

A Complete, Practical Guide to Bit Manipulation Techniques in Python

PART 1: BIT MANIPULATION FUNDAMENTALS

- 1.1 What is Bit Manipulation?
- 1.2 Core Terminology
- 1.3 The Bitwise Operators
- 1.4 Python-Specific Caveats (Negative Numbers & Infinite Bit Width)
- 1.5 How to Read This Handbook

PART 2: TOPICS

- 2.1 Basic Bit Operations — Set, Clear, Toggle, Check
- 2.2 Counting Bits & Brian Kernighan's Algorithm
- 2.3 XOR Properties & Applications
- 2.4 Bit Masking, Subsets & Power Set via Bits
- 2.5 Shifting Tricks — Powers of 2 & Extracting Bits
- 2.6 Arithmetic Without Operators
- 2.7 Single Number / Missing Number Family
- 2.8 Two's Complement, Sign Bit & Overflow
- 2.9 Gray Code & Bit Reversal
- 2.10 Advanced Bitmask Techniques

PART 3: QUICK REFERENCE CHEAT SHEET

Decision Guide — “Which Bit Trick Do I Use?”

PART 1: FUNDAMENTALS

1.1 What is Bit Manipulation?

Bit manipulation means operating directly on the individual binary digits (bits) that make up a number, using bitwise operators (AND, OR, XOR, NOT, shifts) instead of arithmetic or comparison operators. Because a CPU performs bitwise operations in a single cycle, bit tricks are often the fastest possible way to solve a problem, and they can replace loops, extra memory, or even entire data structures (e.g., a bitmask replacing a boolean array) with a single integer and $O(1)$ operations.

1.2 Core Terminology

Bit	A single binary digit, 0 or 1.
Bit Position / Index	The place of a bit within a number, counted from the right starting at 0 (the least significant bit, LSB).
MSB / LSB	Most Significant Bit (leftmost, highest place value) / Least Significant Bit (rightmost, position 0).
Set Bit	A bit with value 1.
Bitmask	An integer used as a set of flags, where each bit position represents membership/state of one item.
Popcount / Hamming Weight	The number of set (1) bits in a number's binary representation.
Hamming Distance	The number of bit positions at which two numbers differ — equals popcount(a XOR b).
Two's Complement	The standard way signed integers are represented in binary, where negation is done by inverting all bits and adding 1.
Overflow	When an arithmetic result doesn't fit in the fixed number of bits allotted to a type (not directly observable in Python's arbitrary-precision integers, but essential for fixed-width languages and for simulating them).

1.3 The Bitwise Operators

& (AND)	Result bit is 1 only if BOTH input bits are 1. Used to check/clear/isolate specific bits.
 (OR)	Result bit is 1 if EITHER input bit is 1. Used to set specific bits.
^ (XOR)	Result bit is 1 if the input bits DIFFER. Used to toggle bits, and has powerful self-canceling properties (see 2.3).
~ (NOT)	Flips every bit (1→0, 0→1). In Python, <code>~x</code> equals <code>-(x+1)</code> due to infinite-width two's complement (see 1.4).
<< (Left Shift)	Shifts bits left, filling with 0s on the right. <code>x << k</code> is equivalent to <code>x * 2^k</code> .

>> (Right Shift)

Shifts bits right. $x \gg k$ is equivalent to $x // 2^k$ for non-negative x (Python uses arithmetic shift, sign-extending for negatives).

1.4 Python-Specific Caveats (Negative Numbers & Infinite Bit Width)

- Python integers have NO fixed width — they behave as if they have infinitely many bits, with negative numbers conceptually having infinite leading 1s (two's complement extended indefinitely).
- This means $\sim x$ in Python is exactly $-(x + 1)$, and bit tricks that rely on a FIXED width (like "flip all 32 bits") need an explicit mask, e.g. $x \wedge 0xFFFFFFFF$ for a 32-bit flip, or the result will look wrong (e.g., unexpectedly negative or huge).
- When simulating fixed-width behavior (common in interview problems that say "as a 32-bit integer"), mask with $\& 0xFFFFFFFF$ after operations, and convert back to a signed value manually if the top bit is set.
- `bin(x)` on a negative number in Python prints a minus sign followed by the binary of the absolute value (NOT two's complement) — don't rely on `bin()` to inspect two's complement bit patterns directly; mask first.

1.5 How to Read This Handbook

For every topic below you will get, in this exact order: (1) What it's for, (2) Use Cases, (3) Limitations, (4) Time & Space Complexity, (5) Approach (short) with the underlying bit-level reasoning, (6) Keywords & Constraints, (7) Fully commented Python code, and (8) 10 Practice Questions each with a short reasoning of why the technique fits.

PART 2: PATTERNS & ALGORITHMS

2.1 Basic Bit Operations — Set, Clear, Toggle, Check

What it's for: The four fundamental single-bit operations every other bit trick is built from: turning a specific bit on (set), turning it off (clear), flipping it (toggle), and reading its current value (check).

Use Cases:

- Implementing a bitmask-based boolean array/flag set (e.g., "visited" set for small n , feature flags).
- Manipulating individual bits of a fixed-width value (permissions, configuration flags, hardware registers conceptually).
- The building block for every bitmask DP problem (see DP handbook, 2.10) and every backtracking bitmask optimization (see Backtracking handbook, 2.8).
- Extracting or modifying specific fields packed into a single integer.

Limitations:

- Bit position must be tracked correctly relative to 0-indexing from the LSB — an extremely common off-by-one source is confusing "the i th bit" with "the i th character from the left" when converting to/from a string.
- These operations don't validate bounds — setting bit 100 on a Python int works fine (Python ints grow arbitrarily), but in fixed-width contexts this would silently do nothing or wrap around.
- Toggle is NOT idempotent — applying it twice returns to the original state, which is easy to forget when toggling inside a loop that might run a variable number of times.

Time Complexity: $O(1)$ per operation **Space Complexity:** $O(1)$

Approach (short): PSEUDOCODE: check bit $i \rightarrow (\text{num} \gg i) \& 1$. set bit $i \rightarrow \text{num} | (1 \ll i)$. clear bit $i \rightarrow \text{num} \& \sim(1 \ll i)$. toggle bit $i \rightarrow \text{num} \wedge (1 \ll i)$. Each builds a "mask" $(1 \ll i)$ isolating just position i , then combines it with the number using the appropriate operator.

Keywords & Constraints: "set/clear/toggle a bit", "check if bit is set", " i th bit", n typically small (bit positions ≤ 32 -64).

```
def check_bit(num, i):
    """Is bit i (0-indexed from LSB) set (1)?"""
    return (num >> i) & 1                # shift bit i down to position 0, mask off everything else

def set_bit(num, i):
    """Turn bit i ON, leave every other bit unchanged."""
    return num | (1 << i)                # OR with a mask that has ONLY bit i set -> forces it to 1

def clear_bit(num, i):
    """Turn bit i OFF, leave every other bit unchanged."""
    return num & ~(1 << i)               # AND with a mask that has EVERY bit set EXCEPT bit i

def toggle_bit(num, i):
    """Flip bit i: 0 -> 1 or 1 -> 0, leave every other bit unchanged."""
    return num ^ (1 << i)                # XOR with a mask that has only bit i set flips just that bit

def update_bit(num, i, value):
    """Set bit i to a specific value (0 or 1) in one operation."""
    cleared = num & ~(1 << i)             # first clear it, regardless of current state
    return cleared | (value << i)         # then OR in the new value shifted to position i

def lowest_set_bit(num):
    """Isolate just the lowest (rightmost) set bit -- a very common building block."""
```

```
return num & (-num)
```

```
# trick: -num in two's complement flips all bits and adds 1
```

Practice Questions — Basic Bit Operations — Set, Clear, Toggle, Check

Q1. Check if the *i*th Bit is Set Determine whether bit *i* of a number is 1. *Approach:* Right-shift the number by *i* positions so that bit lands at position 0, then mask with &1 to read just that bit.

Q2. Set the *i*th Bit Turn on bit *i* without disturbing any other bit. *Approach:* OR the number with a mask that has only bit *i* set ($1 \ll i$) — OR-ing with 1 forces a 1, OR-ing with 0 leaves the bit unchanged.

Q3. Clear the *i*th Bit Turn off bit *i* without disturbing any other bit. *Approach:* AND the number with the complement of ($1 \ll i$), which is all 1s except position *i* — AND-ing with 0 forces a 0, AND-ing with 1 leaves the bit unchanged.

Q4. Toggle the *i*th Bit Flip bit *i* ($0 \rightarrow 1$ or $1 \rightarrow 0$) without disturbing other bits. *Approach:* XOR the number with ($1 \ll i$) — XOR-ing any bit with 1 flips it, XOR-ing with 0 leaves it unchanged.

Q5. Update the *i*th Bit to a Given Value Set bit *i* to exactly 0 or 1 (whichever is given) in a single operation. *Approach:* Clear the bit first (AND with $\sim(1 \ll i)$), then OR in the desired value shifted into position *i* — works regardless of the bit's original state.

Q6. Isolate the Lowest Set Bit Extract just the rightmost 1-bit of a number, zeroing out everything else. *Approach:* $\text{num} \& (-\text{num})$ works because two's complement negation flips all bits and adds 1, which makes every bit above the lowest set bit cancel to 0 when ANDed with the original.

Q7. Clear the Lowest Set Bit Turn off just the rightmost 1-bit, leaving all higher bits unchanged. *Approach:* $\text{num} \& (\text{num} - 1)$ works because subtracting 1 flips the lowest set bit to 0 and every bit below it to 1, so ANDing clears exactly that lowest bit and nothing else.

Q8. Check if Two Numbers Differ by Exactly One Bit Determine if two numbers are identical except for a single flipped bit. *Approach:* XOR the two numbers (differing bits become 1, matching bits become 0), then check the result has EXACTLY one set bit using the Kernighan trick ($\text{num} \& (\text{num} - 1) == 0$).

Q9. Extract a Range of Bits Extract bits from position *i* to *j* (inclusive) as their own number. *Approach:* Right-shift by *i* to align the range to the LSB, then mask with $((1 \ll (j - i + 1)) - 1)$ to keep only the desired number of bits.

Q10. Insert One Number's Bits into Another at a Given Position Insert the bits of number *M* into number *N* starting at bit position *i* through *j*. *Approach:* Clear bits *i* through *j* of *N* using a mask of 0s in that range and 1s elsewhere, then OR in *M* shifted left by *i* — combines clear+set into a range operation.

2.2 Counting Bits & Brian Kernighan's Algorithm

What it's for: Techniques for counting the number of set bits (popcount / Hamming weight) in a number, ranging from the naive check-every-bit approach to the much faster Brian Kernighan trick that only loops once per SET bit.

Use Cases:

- Computing Hamming distance/weight for error-detection, genetics/bioinformatics-style bit comparisons.
- Any problem phrased as "count set bits", "number of 1s in binary representation".
- A building block for bitmask DP (counting how many items are in the current subset).
- Population-count-style aggregation across a range of numbers (Counting Bits DP problem).

Limitations:

- Brian Kernighan's algorithm loops once per SET bit, so it's only faster than the naive 32/64-iteration loop when the number is sparse (few set bits); for a number like all-1s it does the same amount of work as the naive approach.
- Python's built-in `bin(x).count('1')` is often fast enough in practice and simpler to write — hand-rolled bit-counting is mainly valuable for understanding the underlying trick or in languages without a built-in popcount.
- For counting bits across a RANGE of numbers (0 to n), a naive per-number count is $O(n \log n)$ — the DP recurrence (2.10-adjacent) using previously computed counts is $O(n)$ and usually the expected solution.

Time Complexity: $O(\log n)$ naive (check every bit); $O(k)$ with Kernighan's trick, where k = number of set bits

Space Complexity: $O(1)$

Approach (short): PSEUDOCODE (Kernighan): `count = 0; while num: num = num & (num - 1); count += 1`. Each iteration clears exactly the lowest set bit (see 2.1), so the loop runs exactly once per set bit rather than once per total bit position.

Keywords & Constraints: "count set bits", "number of 1 bits", "Hamming weight", "popcount", n up to 32/64-bit integers, or arrays up to 10^5 - 10^6 for range-counting variants.

```
def count_bits_naive(num):
    """Check every bit position --  $O(\log n)$  /  $O(32)$  for a fixed-width int."""
    count = 0
    while num:
        count += num & 1          # check the lowest bit
        num >>= 1                # shift right to check the next bit
    return count

def count_bits_kernighan(num):
    """Brian Kernighan's trick: only loop once per SET bit, not once per total bit position."""
    count = 0
    while num:
        num &= (num - 1)          # clears the lowest set bit each iteration (see 2.1)
        count += 1
    return count

def count_bits_builtin(num):
    """Python's built-in and often fastest option in practice."""
    return bin(num).count('1')

def counting_bits_dp(n):
    """Return popcount for every number 0..n using  $O(n)$  total work via a DP recurrence."""
    dp = [0] * (n + 1)
    for i in range(1, n + 1):
        dp[i] = dp[i >> 1] + (i & 1)    # popcount(i) = popcount(i//2) + (i's lowest bit)
    return dp

def hamming_distance(a, b):
    """Number of bit positions where a and b differ."""
```

```
return count_bits_kernighan(a ^ b)
```

```
# differing bits become 1s after XOR; count them
```

Practice Questions — Counting Bits & Brian Kernighan's Algorithm

Q1. Number of 1 Bits (Hamming Weight) Count the number of set bits in a number's binary representation.

Approach: Repeatedly clear the lowest set bit using $\text{num} \& (\text{num}-1)$ and count iterations until num becomes 0 — Brian Kernighan's trick, $O(k)$ for k set bits.

Q2. Counting Bits for Every Number 0 to N Return an array where $\text{result}[i] = \text{popcount}(i)$ for every i from 0 to n .

Approach: DP recurrence $\text{dp}[i] = \text{dp}[i \gg 1] + (i \& 1)$: the popcount of i equals the popcount of i with its lowest bit removed, plus whether that lowest bit was set.

Q3. Hamming Distance Between Two Numbers Count the number of bit positions where two numbers differ.

Approach: XOR the two numbers so differing bits become 1s and matching bits become 0s, then count the set bits in the result.

Q4. Total Hamming Distance Sum the Hamming distance between every pair of numbers in an array. *Approach:*

Instead of comparing all $O(n^2)$ pairs directly, count how many numbers have each bit position set; for each bit position, contribute $\text{count} \times (n - \text{count})$ pairs that differ at that position (combinatorial reframing avoids the naive pairwise approach).

Q5. Check if a Number is a Power of Two Determine if n is exactly a power of 2 (has exactly one set bit). *Approach:*

A power of 2 has exactly one set bit, so $n \& (n-1) == 0$ (and $n > 0$) confirms it — clearing the lowest set bit leaves nothing only when there was exactly one.

Q6. Check if a Number is a Power of Four Determine if n is exactly a power of 4. *Approach:* Must be a power of 2

(single set bit) AND that set bit must be at an EVEN position — check with $n \& 0xAAAAAAAA == 0$ (a mask of all even-indexed bits) after confirming power-of-2.

Q7. Minimum Bit Flips to Convert One Number to Another Find the minimum number of bit flips to turn a into b .

Approach: Identical to Hamming Distance — XOR the numbers and count the set bits in the result, since each differing bit needs exactly one flip.

Q8. Find the Number with Maximum XOR Value in an Array (Popcount-Adjacent) Among all pairs, find the pair

with maximum popcount of their XOR. *Approach:* Compute $\text{popcount}(a \text{ XOR } b)$ for candidate pairs using a Trie-based greedy bit walk (see Tree handbook, Trie topic, Q7) rather than brute-force $O(n^2)$ pairwise XOR and count.

Q9. Binary Watch (Count Combinations by Popcount) List all valid times where the total set bits across

hours/minutes equals a target count. *Approach:* For each candidate (hour, minute) pair, use popcount (Kernighan or builtin) to check if their combined set-bit count matches the target — direct filtering application of bit counting.

Q10. Sort Integers by the Number of 1 Bits Sort an array by popcount, breaking ties by value. *Approach:* Compute

popcount for every element (builtin or Kernighan's trick), then sort using a composite key of (popcount, value) — shows counting bits as a preprocessing step feeding into a standard sort.

2.3 XOR Properties & Applications

What it's for: XOR (exclusive or) has uniquely useful algebraic properties — it's its own inverse ($x \oplus x = 0$), identity-preserving ($x \oplus 0 = x$), commutative, and associative — which together make it the tool of choice for "find the odd one out", swapping values, and toggling state without extra memory.

Use Cases:

- Finding a single number that appears an odd number of times while everything else appears an even number of times (Single Number family, see 2.7).
- Swapping two variables without a temporary variable.
- Detecting differences between two datasets/bitstrings (checksums, simple error detection).
- Encryption basics (XOR cipher) and toggling/flipping state compactly.

Limitations:

- XOR-swap (no temp variable) fails silently if the two variables are actually the SAME memory location (e.g., $a[i]$, $a[i]$ with $i=j$) — it zeroes the value out instead of leaving it unchanged. Always guard against aliasing.
- The "find the unique element" XOR trick only works when every OTHER element appears an EVEN number of times; if elements can appear an odd number of times more than once, XOR alone can't distinguish between multiple candidates without extra partitioning logic (see Single Number III in 2.7).
- XOR cipher-style "encryption" is trivially breakable with known-plaintext attacks — useful as a bit-manipulation exercise, not as real security.

Time Complexity: $O(n)$ — one XOR per element for most applications **Space Complexity:** $O(1)$

Approach (short): PSEUDOCODE: $result = 0$; for each num in array: $result \oplus= num$. Because $a \oplus a = 0$ and $a \oplus 0 = a$, every value that appears an even number of times cancels itself out, leaving only values that appear an odd number of times.

Keywords & Constraints: "appears exactly once, others appear twice", "swap without temp variable", "find the missing/duplicate number using XOR", $n \leq 10^5$ - 10^6 .

```
def find_single_number(nums):
    """Every element appears twice except one -- find that one using XOR's self-cancellation."""
    result = 0
    for num in nums:
        result ^= num
    return result
    # pairs cancel to 0 (a^a=0), leftover is the unique value

def swap_without_temp(a, b):
    """Swap two values using XOR, no temporary variable needed."""
    a = a ^ b
    b = a ^ b
    a = a ^ b
    return a, b
    # b becomes original a (since a is now a^b, and b^(a^b)=a)
    # a becomes original b

def missing_number(nums):
    """Array contains n distinct numbers from 0..n with exactly one missing -- find it via XOR."""
    result = len(nums)
    for i, num in enumerate(nums):
        result ^= i ^ num
    return result
    # start with n itself (the "extra" index not in the array)
    # XOR in both the index and the value
    # everything cancels except the missing number

def find_two_single_numbers(nums):
    """Exactly TWO elements appear once, everything else appears twice -- find both."""
    xor_all = 0
    for num in nums:
        xor_all ^= num
    # xor_all = a ^ b (the two unique numbers)
```



```

# find any set bit in xor_all -- a and b MUST differ at this bit (since xor_all has it set)
diff_bit = xor_all & (-xor_all) # isolate lowest set bit (see 2.1/2.2)

a = b = 0
for num in nums:
    if num & diff_bit:
        a ^= num # partition all numbers into two groups by that bit
    else:
        b ^= num
return a, b # each group's XOR isolates one of the two unique numbers

def xor_cipher(text, key):
    """Simple (INSECURE) XOR cipher -- applying it twice with the same key restores the original."""
    return ''.join(chr(ord(ch) ^ key) for ch in text)

```

Practice Questions — XOR Properties & Applications

Q1. Single Number Find the element that appears once while every other element appears exactly twice. *Approach:* XOR every element together — pairs cancel to 0 ($a \oplus a = 0$), leaving only the unique value ($x \oplus 0 = x$).

Q2. Single Number III Find the two elements that each appear once while everything else appears twice. *Approach:* XOR everything to get $a \oplus b$, isolate any set bit in that result (a and b must differ there), then partition all numbers by that bit into two groups and XOR each group separately to isolate a and b.

Q3. Missing Number Find the missing number in an array containing n distinct numbers from the range 0 to n. *Approach:* XOR every index $0..n-1$, every array value, and n itself together — every present number cancels with its matching index, leaving only the missing number.

Q4. Swap Two Numbers Without a Temporary Variable Swap the values of two variables using only XOR. *Approach:* $a \oplus b$; $b \oplus a$; $a \oplus b$ — each step algebraically recovers the other original value using XOR's self-inverse property; must guard against a and b being the same memory location.

Q5. Find the Duplicate Number Using XOR Find the single duplicate in an array of $n+1$ integers where values range from 1 to n. *Approach:* XOR all array values together with all numbers $1..n$ — every number that appears exactly once cancels, leaving the number that appeared twice (assuming exactly one duplicate and one missing arrangement, adapt condition per exact problem constraints).

Q6. Complement of a Number Find the bitwise complement of a number (flip all bits within its own bit-length). *Approach:* XOR the number with a mask of all-1s matching its exact bit length (e.g., $(1 \ll \text{bit_length}) - 1$) — flips every relevant bit without affecting higher unused bits the way a raw ~ would in a fixed-width context.

Q7. Maximum XOR of Two Numbers in an Array Find the maximum possible XOR value between any two numbers in an array. *Approach:* Build a binary Trie bit by bit (MSB to LSB) and greedily walk it for each number, always preferring the opposite bit at each level to maximize the running XOR (see Tree handbook Trie section for the full Trie-based technique).

Q8. XOR Queries of a Subarray Answer multiple queries asking for the XOR of a subarray range. *Approach:* Precompute a prefix-XOR array ($\text{prefixXor}[i] = \text{XOR of all elements up to } i$); any range XOR is $\text{prefixXor}[\text{right}] \oplus \text{prefixXor}[\text{left}-1]$, a direct XOR analogue of prefix sums.

Q9. Decode XORed Array Given an XOR-encoded array and the first original element, recover the original array. *Approach:* Each original element is recoverable as $\text{original}[i] = \text{encoded}[i-1] \oplus \text{original}[i-1]$, since $\text{encoded}[i] = \text{original}[i] \oplus \text{original}[i+1]$ by definition — walk forward XOR-ing consecutively.

Q10. Find the Original Array of a Doubled Array Given an array where every element's double also appears, reconstruct the original halved array. *Approach:* Not solved by XOR alone (values aren't necessarily paired as duplicates), but included to contrast: sort and greedily match each value with its double using a frequency map — shows recognizing when XOR does NOT apply is equally important as knowing when it does.

2.4 Bit Masking, Subsets & Power Set via Bits

What it's for: Represents a SET of items as a single integer, where bit i being 1 means "item i is in the set". Since an n -item set has exactly 2^n possible subsets, iterating every integer from 0 to $2^n - 1$ directly enumerates every possible subset without any recursion.

Use Cases:

- Generating all subsets of a small set ($n \leq \sim 20$) without recursion/backtracking.
- Representing "visited/used" state compactly for bitmask DP (see DP handbook, 2.10) and backtracking optimizations (see Backtracking handbook, 2.8).
- Fast set operations: union ($|$), intersection ($\&$), difference ($\& \sim$), symmetric difference ($\wedge$), all $O(1)$ on the whole set at once.
- Iterating all sub-masks of a given mask (useful in advanced bitmask DP, see 2.10).

Limitations:

- Only practical for $n \leq \sim 20$ -25, since 2^n masks must fit comfortably in memory/time — beyond that, this representation and any full-enumeration approach becomes infeasible.
- Bit position in the mask must consistently map to the SAME item throughout a solution — mixing up which index maps to which bit is a very common source of bugs.
- Enumerating ALL sub-masks of a mask naively (checking every $0..mask$) is wasteful; the "submask enumeration trick" (see 2.10) is needed for efficiency when this appears inside a larger loop.

Time Complexity: $O(2^n)$ to enumerate all subsets; $O(1)$ per set operation (membership check, union, intersection)

Space Complexity: $O(1)$ per mask (a single integer represents an entire subset)

Approach (short): PSEUDOCODE: for mask in range(2^n): subset = [items[i] for i in range(n) if mask & (1 << i)]. Each integer from 0 to $2^n - 1$, read in binary, directly encodes exactly one subset via its set bits.

Keywords & Constraints: "all subsets", "power set", "which items are included/used", $n \leq 20$ -25 (explicit small constraint is a strong signal).

```
def all_subsets_bitmask(items):
    """Generate all subsets of `items` by iterating every possible bitmask 0..2^n - 1."""
    n = len(items)
    result = []
    for mask in range(1 << n):
        # 1 << n == 2^n, every possible subset encoding
        subset = [items[i] for i in range(n) if mask & (1 << i)]
        result.append(subset)
    return result

def is_item_in_subset(mask, i):
    """Membership check: is item i part of the subset encoded by mask?"""
    return (mask >> i) & 1 == 1

def union(mask_a, mask_b):
    return mask_a | mask_b
    # item is in the union if in EITHER set

def intersection(mask_a, mask_b):
    return mask_a & mask_b
    # item is in the intersection if in BOTH sets

def difference(mask_a, mask_b):
    return mask_a & ~mask_b
    # items in A but NOT in B

def symmetric_difference(mask_a, mask_b):
    return mask_a ^ mask_b
    # items in exactly one of A, B (not both)

def count_subsets_with_property(items, is_valid):
    """Generic template: count subsets satisfying an arbitrary predicate, via full mask enumeration."""
```

```

n = len(items)
count = 0
for mask in range(1 << n):
    subset = [items[i] for i in range(n) if mask & (1 << i)]
    if is_valid(subset):
        count += 1
return count

```

Practice Questions — Bit Masking, Subsets & Power Set via Bits

Q1. Generate All Subsets Using Bitmasks Return the power set of a set without recursion. *Approach:* Loop every integer 0 to $2^n - 1$; each integer's binary representation directly encodes one subset via its set bits.

Q2. Count Subsets with a Given Sum Count how many subsets of an array sum to a target (small n). *Approach:* Enumerate every bitmask, build the corresponding subset, sum it, and check against the target — brute-force but simple for $n \leq \sim 20$ (compare with the DP subset-sum approach from the DP handbook for larger n).

Q3. Maximum AND Sum of Array (Bitmask Over Assignment Slots) Assign array elements to numbered slots maximizing sum of element AND slot-bitmask. *Approach:* Bitmask represents which slots are currently filled; DP over (element index, slot-usage mask) tries assigning the current element to each available slot pair.

Q4. Partition into Two Subsets with Minimum Difference Split an array into two subsets minimizing the absolute difference of their sums. *Approach:* For small n , enumerate all masks representing "subset A", compute its sum, and track the mask minimizing $|\text{sum}(A) - (\text{total} - \text{sum}(A))|$ — contrast with the DP/knapsack approach for larger n .

Q5. Check if Array Can Be Divided into Pairs with Equal Sum (Small N) Determine if array elements can be paired up so every pair has the same sum. *Approach:* Enumerate masks representing which elements have been paired so far, recursively/iteratively trying to extend a valid pairing — bitmask state tracks partial pairing progress.

Q6. Smallest Sufficient Team Find the smallest team of people covering all required skills, each person having a subset of skills (bitmask). *Approach:* DP over skill-bitmask: $\text{dp}[\text{skill_mask}] = \text{smallest team covering exactly those skills}$, trying each person as a candidate addition that OR-extends the current skill mask.

Q7. Word Break Using Bitmask (Small String) Determine if a short string can be segmented into dictionary words, using bitmask to track split points. *Approach:* $\text{dp}[\text{mask}]$ where each bit represents whether that boundary position is a valid split point, propagating forward by checking valid dictionary substrings between set bits.

Q8. Maximum Students Taking Exam (Seat Bitmask) Arrange students in a grid of seats (some broken) to maximize count with no adjacent cheating. *Approach:* Represent each row's seating arrangement as a bitmask; DP transitions between valid row-masks (no two adjacent bits set, and no conflicting diagonal bits with the previous row's mask).

Q9. Number of Subsets with Bitwise OR Equal to K Count subsets of an array whose bitwise OR equals exactly K . *Approach:* For small n , enumerate every mask, compute the OR of the corresponding subset via a fold/reduce, and compare to K — direct brute-force bitmask enumeration application.

Q10. Minimum XOR Sum of Two Arrays (Bitmask Assignment) Pair up elements from two arrays (one-to-one) to minimize the sum of pairwise XORs. *Approach:* DP over a bitmask representing which elements of the second array have already been used, trying every unused element as the pairing partner for the next element of the first array.

2.5 Shifting Tricks — Powers of 2 & Extracting Bits

What it's for: Left and right shifts ($\ll$, $\gg$) are the fastest way to multiply/divide by powers of 2, and are the primary tool for moving bits into or out of specific positions — essential for building masks, packing/unpacking values, and any power-of-2-related check or computation.

Use Cases:

- Fast multiplication/division by powers of 2 ($x \ll k == x * 2^k$, $x \gg k == x // 2^k$ for non-negative x).
- Building bitmasks of a specific width, e.g., $(1 \ll n) - 1$ gives n set bits (all 1s).
- Packing multiple small values into a single integer (e.g., RGB color channels, coordinate pairs) and unpacking them back out.
- Binary search over the answer combined with bit-by-bit construction (e.g., building the answer bit by bit from MSB to LSB).

Limitations:

- Right-shifting a **NEGATIVE** number in Python performs an arithmetic shift (sign-extending with 1s), which differs from a logical shift — be careful when a problem implicitly assumes unsigned/fixed-width behavior.
- Shifting by an amount $\geq$ the type's bit width is undefined behavior in many languages (though Python, with arbitrary precision, just keeps growing/shrinking the number correctly) — don't assume Python's forgiving behavior translates to other languages.
- $(1 \ll n) - 1$ to build an n -bit all-1s mask is correct, but forgetting the "- 1" is an extremely common bug (that gives a mask with a single high bit set instead of n low bits set).

Time Complexity: $O(1)$ per shift operation **Space Complexity:** $O(1)$

Approach (short): PSEUDOCODE: multiply by $2^k \rightarrow x \ll k$. divide by $2^k \rightarrow x \gg k$. build a k -bit all-1s mask $\rightarrow (1 \ll k) - 1$. extract the top k bits of an n -bit number $\rightarrow \text{num} \gg (n - k)$.

Keywords & Constraints: "multiply/divide by power of 2", "build a mask of n bits", "pack/unpack values", fixed small bit-widths (8/16/32/64).

```
def multiply_by_power_of_2(x, k):
    return x << k                                # equivalent to x * (2 ** k), but O(1) hardware-level shift

def divide_by_power_of_2(x, k):
    return x >> k                                # equivalent to x // (2 ** k) for non-negative x

def build_n_bit_mask(n):
    """A mask with the lowest n bits set to 1, everything above is 0."""
    return (1 << n) - 1                          # e.g. n=4: (1<<4)=0b10000, minus 1 = 0b01111

def pack_two_16bit_values(high, low):
    """Pack two 16-bit values into a single 32-bit integer."""
    return (high << 16) | low                    # shift 'high' into the upper half, OR in 'low' unchanged

def unpack_two_16bit_values(packed):
    """Reverse of pack_two_16bit_values."""
    high = packed >> 16                          # shift the upper half back down
    low = packed & 0xFFFF                        # mask off just the lower 16 bits
    return high, low

def next_power_of_two(n):
    """Smallest power of 2 >= n, via shifting."""
    if n <= 1:
        return 1
    power = 1
```

```

while power < n:
    power <= 1
    return power
# keep doubling until we reach/exceed n

def is_power_of_two(n):
    return n > 0 and (n & (n - 1)) == 0
# single set bit check (see 2.2)

```

Practice Questions — Shifting Tricks — Powers of 2 & Extracting Bits

Q1. Multiply a Number by 2 Without Using * Double a number using bit shifting. *Approach:* $x \ll 1$ is exactly equivalent to $x * 2$, since a left shift by 1 moves every bit one place value higher.

Q2. Divide a Number by 2 Without Using / Halve a number (integer division) using bit shifting. *Approach:* $x \gg 1$ is exactly equivalent to $x // 2$ for non-negative x , discarding the remainder bit that shifts out.

Q3. Build a Mask with the Lowest N Bits Set Construct an integer with exactly the lowest n bits set to 1. *Approach:* $(1 \ll n) - 1$: shifting 1 into position n gives a single bit above the desired range, subtracting 1 borrows through all lower positions turning them all to 1.

Q4. Extract the Top K Bits of an N-Bit Number Get just the highest k bits of a number, given its total bit width n . *Approach:* Right-shift by $(n - k)$ to discard the lower $(n - k)$ bits, leaving only the top k bits in the low positions.

Q5. Find the Next Power of Two Greater Than or Equal to N Find the smallest power of 2 that is $\geq n$. *Approach:* Repeatedly left-shift starting from 1 until reaching or exceeding n — or use the faster "smear" trick: OR the number with itself shifted right by 1,2,4,8,16, then add 1.

Q6. Round Down to the Nearest Power of Two Find the largest power of 2 that is $\leq n$. *Approach:* Isolate the highest set bit: repeatedly clear the lowest set bit $(n \& (n-1))$ until only one bit remains, or compute $1 \ll (n.bit_length() - 1)$.

Q7. Pack Two Values into a Single Integer Combine two smaller integers into one larger integer for compact storage. *Approach:* Left-shift the first value into the upper bits and OR in the second value in the lower bits, reversing with a right-shift and a mask to unpack.

Q8. Reverse the Order of Two Halves of a Byte Swap the upper 4 bits and lower 4 bits of an 8-bit number. *Approach:* Extract each nibble with shifting and masking $((x \gg 4) \& 0xF$ for the upper, $x \& 0xF$ for the lower), then reassemble them shifted into each other's original position.

Q9. Find the Position of the Only Set Bit Given a number known to be a power of 2, find which bit position is set. *Approach:* Repeatedly right-shift the number counting iterations until it becomes 1, or use $\text{num.bit_length()} - 1$ which directly gives the 0-indexed position of the highest (and here, only) set bit.

Q10. Binary Search Combined with Bit Construction (Kth Bit Set Query) Build up an answer bit by bit from the most significant bit down, testing each candidate bit against a feasibility check. *Approach:* Start with $\text{answer} = 0$; for each bit position from high to low, tentatively set that bit in the answer and check feasibility (via a problem-specific test), keeping the bit set only if it still satisfies constraints — a general "greedy bit-by-bit construction" pattern common in advanced bit problems.

2.6 Arithmetic Without Operators

What it's for: Performs addition, subtraction, or other arithmetic using ONLY bitwise operators, by directly simulating how binary addition/subtraction works at the hardware level (with carries and borrows).

Use Cases:

- Classic interview questions testing deep understanding of how arithmetic works at the bit level ("add two numbers without using +").
- Implementing arithmetic in constrained environments where the + operator is unavailable or overloaded differently.
- Understanding how a CPU's ALU (Arithmetic Logic Unit) actually computes addition using full-adder circuits (XOR for sum, AND for carry).
- A stepping stone to understanding two's complement negation (2.8) and overflow detection.

Limitations:

- These techniques are almost never the PRACTICAL way to do arithmetic (the built-in + is faster and clearer) — their value is educational/interview-specific, demonstrating bit-level fluency.
- The addition-via-XOR-and-carry loop can be tricky to bound correctly in a fixed-width (e.g., 32-bit) context due to how carries propagate and potentially overflow — needs explicit masking in languages/contexts simulating fixed width.
- Python's arbitrary-precision integers mean these tricks "just work" without overflow concerns here, but that masks an important difference from fixed-width languages where the same code needs extra care.

Time Complexity: $O(\log n)$ — bounded by the number of bits, since each iteration eliminates at least one bit of carry

Space Complexity: $O(1)$

Approach (short): PSEUDOCODE (addition): while carry_bits != 0: sum_without_carry = a ^ b (adds bits ignoring carry); carry = (a & b) << 1 (computes the carry from each bit position, shifted to where it needs to be added); a, b = sum_without_carry, carry; repeat until there's no carry left.

Keywords & Constraints: "add two numbers without using +", "swap without temp", "negate a number using bits", "multiply without *", interview-style constrained-operator problems.

```
def add_without_plus(a, b):
    """Add two integers using only XOR (sum without carry) and AND+shift (carry), simulating a full adder."""
    while b != 0:
        sum_without_carry = a ^ b                # XOR adds corresponding bits, ignoring any carry
        carry = (a & b) << 1                      # AND finds positions where BOTH bits are 1 (a carry occurs),
                                                # shifted left because a carry affects the NEXT higher bit
        a, b = sum_without_carry, carry
    return a

def negate_number(x):
    """Negate a number using two's complement: flip all bits, then add 1."""
    return add_without_plus(~x, 1)                # ~x flips every bit, +1 completes the two's complement negation

def subtract_without_minus(a, b):
    """Subtract by adding the negation."""
    return add_without_plus(a, negate_number(b))

def multiply_without_star(a, b):
    """Multiply using shift-and-add (mirrors how binary long multiplication works)."""
    result = 0
    negative = (a < 0) != (b < 0)                # track sign separately, work with absolute values
    a, b = abs(a), abs(b)
    while b:
        if b & 1:                                # if the lowest bit of b is set, add the current shifted a
            result = add_without_plus(result, a)
        a <<= 1                                  # double a (shift left) as we move to the next bit of b
        b >>= 1
```

```

        b >>= 1
    return -result if negative else result

```

```

# move to the next bit of b

```

```

def increment_without_plus(x):
    return add_without_plus(x, 1)

```

Practice Questions — Arithmetic Without Operators

Q1. Add Two Numbers Without Using + Compute the sum of two integers using only bitwise operators. *Approach:* XOR gives the sum ignoring carries, AND+left-shift gives the carry bits; repeat, feeding the carry back in, until there's no carry left — directly simulates a hardware full adder.

Q2. Subtract Two Numbers Without Using - Compute the difference of two integers using only bitwise operators. *Approach:* Negate the second number (flip all bits, add 1 — two's complement) and then add, reducing subtraction to the addition trick.

Q3. Negate a Number Without Using - Compute $-x$ using only bitwise operators. *Approach:* Two's complement definition: flip every bit of x ($\sim x$) and add 1 using the bitwise addition trick.

Q4. Multiply Two Numbers Without Using * Compute the product of two integers using only bitwise operators. *Approach:* Shift-and-add: for each set bit in one operand, add the other operand shifted to that bit's position — mirrors binary long multiplication by hand.

Q5. Divide Two Numbers Without Using / Compute integer division of two numbers using only bitwise operators. *Approach:* Repeatedly find the largest shifted multiple of the divisor that still fits within the remaining dividend, subtract it (via bitwise subtraction), and accumulate the corresponding shift amount into the quotient.

Q6. Increment a Number Without Using + Add 1 to a number using only bitwise operators. *Approach:* Direct special case of the addition trick with $b=1$ — or observe it's equivalent to flipping trailing 1s to 0s and the first trailing 0 to a 1.

Q7. Compute Absolute Value Without Branching Find $|x|$ using only bitwise operators, no if-statement. *Approach:* $\text{mask} = x \gg 31$ (arithmetic shift: all 1s if x is negative, all 0s if non-negative, for a 32-bit int); $(x \wedge \text{mask}) - \text{mask}$ gives the absolute value by conditionally negating via XOR and subtraction, all branchless.

Q8. Swap Two Numbers Without Using a Temp Variable (Arithmetic Version) Swap two variables using addition/subtraction instead of XOR. *Approach:* $a = a + b$; $b = a - b$; b now holds the original a ; $a = a - b$ recovers the original b — contrast with the XOR-based swap (2.3) to see two different techniques solving the same problem.

Q9. Check if Adding Two Numbers Causes Overflow (Simulated 32-bit) Detect if $a + b$ would overflow a signed 32-bit integer. *Approach:* Compute the sum using the bitwise addition trick, then check if the sign bits of the two operands MATCH each other but DIFFER from the result's sign bit — that specific pattern is the signature of signed overflow.

Q10. Compute Average of Two Numbers Without Overflow Find $(a+b)/2$ avoiding the intermediate overflow that a naive $(a+b)/2$ might cause in fixed-width arithmetic. *Approach:* Use $a + (b - a) / 2$ conceptually, or the bit trick $(a \& b) + ((a \wedge b) \gg 1)$, which computes the average by separating the common bits from the bits that need to be "split" via a halved XOR.

2.7 Single Number / Missing Number Family

What it's for: A family of "find the odd one out" problems solvable in $O(n)$ time and $O(1)$ space using bit tricks (mainly XOR, sometimes bit-counting) instead of a hashmap — the signature of this family is that a hashmap solution is easy and obvious, but a strictly $O(1)$ -space bit-based solution is also possible and often what's actually being tested.

Use Cases:

- Finding a unique element among duplicates (Single Number I/II/III).
- Finding a missing or duplicate number in a range-constrained array.
- Any "find the number that doesn't fit the pattern" problem with an $O(1)$ -space constraint hint.
- A common warm-up/building-block before more advanced bitmask problems.

Limitations:

- Only works cleanly when the "everyone else" pattern is REGULAR (all appear exactly twice, or exactly three times) — irregular patterns (some appear twice, others three times) need per-bit counting with modular arithmetic instead of plain XOR.
- When elements can appear THREE times (not two), plain XOR fails since XOR-ing a value with itself 3 times returns the value itself, not 0 — needs per-bit counting mod 3 instead (see code).
- These tricks assume the array actually satisfies the stated pattern; malformed/adversarial input (not matching the assumed frequency pattern) will silently produce a wrong answer with no error.

Time Complexity: $O(n)$ — typically 1-2 passes over the array **Space Complexity:** $O(1)$ — the key selling point over an $O(n)$ -space hashmap solution

Approach (short): PSEUDOCODE (appears once, rest twice): XOR everything. PSEUDOCODE (appears once, rest three times): for each of the 32 bit positions, sum how many numbers have that bit set, take that sum mod 3 — the bits where the remainder is 1 belong to the unique number.

Keywords & Constraints: "every element appears twice except one", "every element appears three times except one", " $O(1)$ space" explicitly required, $n \leq 10^5$ - 10^6 .

```
def single_number_i(nums):
    """Every element appears TWICE except one -- appears ONCE."""
    result = 0
    for num in nums:
        result ^= num
    return result
    # pairs cancel via XOR self-inverse property

def single_number_ii(nums):
    """Every element appears THREE times except one -- appears ONCE. Plain XOR does NOT work here."""
    ones = twos = 0
    for num in nums:
        ones = (ones ^ num) & ~twos
        twos = (twos ^ num) & ~ones
    return ones
    # track bits seen exactly once so far (state machine encoding)
    # track bits seen exactly twice so far
    # after processing all, 'ones' holds bits of the answer

def single_number_ii_bitcount(nums):
    """Alternative, more intuitive version: count set bits at each position, mod 3."""
    result = 0
    for i in range(32):
        bit_sum = sum((num >> i) & 1 for num in nums)
        if bit_sum % 3 != 0:
            result |= (1 << i)
    return result
    # assume 32-bit integers
    # this bit belongs to the unique number

def missing_number(nums):
    """Array has n distinct numbers from range [0, n], exactly one missing."""
```



```

    result = len(nums)
    for i, num in enumerate(nums):
        result ^= i ^ num
    return result

def find_duplicate_and_missing(nums):
    """Array of 1..n with one number replaced by a duplicate of another (one missing, one duplicated)."""
    n = len(nums)
    xor_all = 0
    for i in range(1, n + 1):
        xor_all ^= i
    for num in nums:
        xor_all ^= num
    # xor_all now = missing ^ duplicate
    diff_bit = xor_all & (-xor_all)
    x = y = 0
    for i in range(1, n + 1):
        if i & diff_bit:
            x ^= i
        else:
            y ^= i
    for num in nums:
        if num & diff_bit:
            x ^= num
        else:
            y ^= num
    # x and y are {missing, duplicate} in some order -- one more pass determines which is which
    if x in nums:
        return x, y
    return y, x

```

Practice Questions — Single Number / Missing Number Family

Q1. Single Number Find the element that appears once, every other element appears exactly twice. *Approach:* XOR all elements together — pairs cancel to 0, the unique value survives.

Q2. Single Number II Find the element that appears once, every other element appears exactly THREE times. *Approach:* Plain XOR fails here (XOR-ing a value 3 times returns the value itself); use per-bit counting mod 3, or the compact "ones/twos" state-machine bitwise trick.

Q3. Single Number III Find the two elements that each appear once, every other element appears exactly twice. *Approach:* XOR everything to get $a \oplus b$, isolate a distinguishing set bit, then partition the array by that bit into two groups and XOR each group separately.

Q4. Missing Number Find the missing number in an array of n distinct numbers from range $[0, n]$. *Approach:* XOR every index and every array value together with n itself — present numbers cancel with matching indices, leaving only the missing number.

Q5. Find the Duplicate Number Find the one duplicated number in an array of $n+1$ integers from range $[1, n]$. *Approach:* Cycle detection (Floyd's) is the standard $O(1)$ -space solution here rather than pure XOR (since exactly one number repeats an unknown number of times) — included to show not every "find the odd one" problem is a pure bit-trick, some need a different $O(1)$ -space technique.

Q6. Set Mismatch (Find the Duplicate AND the Missing Number) An array of $1..n$ has one number replaced by a duplicate of another; find both the duplicate and the missing number. *Approach:* XOR everything ($1..n$ and the array) to get $\text{duplicate} \oplus \text{missing}$, isolate a distinguishing bit, partition into two groups, XOR each to recover both numbers, then one membership check determines which is which.

Q7. Find the Element Appearing Odd Number of Times Among elements each appearing an even number of times except one appearing an odd number of times, find it. *Approach:* Direct generalization of Single Number I — XOR everything, since any value appearing an even number of times cancels itself out regardless of exactly how many times (2, 4, 6...).

Q8. Find Two Numbers Appearing Odd Number of Times Two elements appear an odd number of times, all others appear an even number of times; find both. *Approach:* Same technique as Single Number III — XOR everything to isolate $a \oplus b$, split by a distinguishing bit, and XOR each partition separately.

Q9. Find the Non-Repeating Element in a Sorted Array (All Others Appear Twice) In a sorted array where every element appears twice except one, find the unique one in $O(\log n)$. *Approach:* Since the array is sorted, pairs align at (even, odd) index boundaries until the unique element appears, after which the pattern shifts — binary search on this parity shift beats the $O(n)$ XOR scan by exploiting the sortedness (a good example of combining bit-trick thinking with a completely different technique, binary search, when extra structure is available).

Q10. XOR of All Numbers in a Given Range Compute the XOR of all integers from 1 to n efficiently, without looping. *Approach:* XOR from 1 to n follows a repeating pattern based on $n \bmod 4$ (n , 1, $n+1$, or 0) — a closed-form $O(1)$ formula derived by observing how consecutive XORs cancel in groups of 4, useful as a building block for range-XOR queries.

2.8 Two's Complement, Sign Bit & Overflow

What it's for: The standard binary representation for signed integers, where the most significant bit indicates sign and negative numbers are represented so that ordinary binary addition "just works" for both positive and negative values without special-casing subtraction.

Use Cases:

- Simulating fixed-width (8/16/32/64-bit) signed integer behavior in a language (like Python) that doesn't have it natively.
- Detecting/handling integer overflow in problems that explicitly constrain results to a fixed bit width (very common in interview problems: "assume a 32-bit signed integer").
- Understanding why $-x == -(x+1)$, why right-shifting a negative number sign-extends, and other "surprising" bit behaviors around negative numbers.
- Bit manipulation problems that explicitly involve negative numbers (reversing bits of a negative number, converting between signed/unsigned representations).

Limitations:

- Python integers are NOT fixed-width, so two's complement effects that are automatic in C/Java/etc. must be manually simulated with explicit masking ($\& 0xFFFFFFFF$ for 32-bit) — forgetting this is the single biggest source of bugs when porting bit-manipulation problems into Python.
- Converting a masked Python int back to its "intended" signed value requires an explicit check-and-subtract step (if $\text{value} \geq 2^{31}$: $\text{value} -= 2^{32}$ for 32-bit) — Python won't do this automatically.
- Overflow detection logic is inherently a bit fiddly (comparing sign bits of operands vs. result) and easy to get subtly wrong — test against known edge cases ($\text{INT_MAX} + 1$, $\text{INT_MIN} - 1$) explicitly.

Time Complexity: $O(1)$ for sign/overflow checks on a fixed-width number **Space Complexity:** $O(1)$

Approach (short): PSEUDOCODE (simulate 32-bit signed result in Python): $\text{result} = \text{raw_computation} \& 0xFFFFFFFF$ (mask down to 32 bits); if $\text{result} \geq 2^{31}$: $\text{result} -= 2^{32}$ (reinterpret the top bit as the sign, converting from Python's unsigned-looking masked value to the true signed value).

Keywords & Constraints: "32-bit signed integer", "overflow", "reverse bits", "convert to two's complement", explicit bit-width mentioned in the problem (8/16/32/64-bit).

```
MASK_32 = 0xFFFFFFFF                # mask for exactly 32 bits
INT_MAX_32 = 2**31 - 1
INT_MIN_32 = -2**31

def to_signed_32(value):
    """Reinterpret a masked (unsigned-looking) 32-bit value as its signed two's-complement equivalent."""
    value &= MASK_32                    # ensure we're only looking at 32 bits
    if value >= 2**31:                  # top bit set -> this represents a negative number
        value -= 2**32                  # reinterpret: subtract 2^32 to get the true negative value
    return value

def simulated_32bit_add(a, b):
    """Add two numbers, simulating 32-bit signed overflow wraparound (like C/Java would do)."""
    result = (a + b) & MASK_32          # raw addition, then truncate to 32 bits
    return to_signed_32(result)

def would_overflow_on_add(a, b):
    """Detect if a + b overflows a 32-bit signed integer, using sign-bit comparison."""
    result = a + b
    return result > INT_MAX_32 or result < INT_MIN_32

def reverse_bits_32(n):
```

```

"""Reverse the bits of a 32-bit unsigned integer."""
result = 0
for i in range(32):
    bit = (n >> i) & 1
    result |= (bit << (31 - i))
return result
# extract bit i from the original (from the LSB side)
# place it at the MIRRORED position in the result

def get_sign_bit(n, bit_width=32):
    """Extract just the sign bit of an n-bit number (1 = negative in two's complement, 0 = non-negative)."""
    return (n >> (bit_width - 1)) & 1

```

Practice Questions — Two's Complement, Sign Bit & Overflow

Q1. Convert a Masked Value to its True Signed Representation Given a value known to represent a 32-bit two's complement number, but stored as a plain non-negative Python int, recover its true signed value. *Approach:* If the value's top bit (bit 31) is set, it represents a negative number in two's complement — subtract 2^{32} to reinterpret it correctly.

Q2. Simulate 32-bit Signed Integer Overflow Add two numbers and return the result as it would appear in a 32-bit signed integer (wrapping on overflow). *Approach:* Mask the raw sum to 32 bits with `& 0xFFFFFFFF`, then reinterpret the top bit as a sign bit if set, subtracting 2^{32} to get the correctly wrapped signed result.

Q3. Reverse Bits of a 32-bit Integer Reverse the bit order of a 32-bit unsigned integer. *Approach:* For each bit position *i* in the original, place it at the mirrored position $(31 - i)$ in the result, built up by OR-ing in each repositioned bit.

Q4. Detect Integer Overflow Before It Happens Check if adding two 32-bit signed integers would overflow, before actually computing the (potentially incorrect) sum. *Approach:* Compare the sign bits: overflow occurs specifically when both operands have the SAME sign but the result would have the OPPOSITE sign — checked via sign-bit comparison rather than computing the sum first.

Q5. Get the Sign Bit of a Number Extract just the sign bit (bit 31 for a 32-bit int) to determine if a number is negative. *Approach:* Right-shift the number by $(\text{bit_width} - 1)$ positions, aligning the sign bit to position 0, then mask with `&1`.

Q6. Absolute Value Using Two's Complement Trick Compute $|x|$ purely via bit manipulation, using the sign bit. *Approach:* Arithmetic-right-shift by 31 to get a mask of all 1s (if negative) or all 0s (if non-negative), then $(x \wedge \text{mask}) - \text{mask}$ conditionally negates *x* exactly when it was negative — branchless.

Q7. Reverse an Integer (LeetCode-style, with Overflow Check) Reverse the digits of a signed 32-bit integer, returning 0 if the result overflows. *Approach:* Build the reversed value digit by digit (arithmetic, not pure bit-shift, since it's base-10 digits), checking after each step whether the running result has exceeded the 32-bit signed range and returning 0 immediately if so.

Q8. Convert Integer to Binary String (Two's Complement for Negatives) Print the 32-bit two's complement binary representation of a (possibly negative) integer. *Approach:* Mask the number with `& 0xFFFFFFFF` to get its unsigned bit pattern representation, then format that masked value as a 32-character binary string.

Q9. Bitwise AND of Numbers Range Find the bitwise AND of all numbers in a range $[m, n]$. *Approach:* The result is the common binary prefix shared by *m* and *n* (any differing lower bits get AND-ed to 0 somewhere in the range) — find it by right-shifting both *m* and *n* together until they're equal, then left-shift back by the same amount.

Q10. Check if Adding Causes Sign Change Unexpectedly (Edge Case Testing) Given `INT_MAX` and `INT_MIN` style edge values, verify overflow detection logic handles them correctly. *Approach:* Explicitly test `would_overflow_on_add` against known boundary pairs (`INT_MAX_32 + 1`, `INT_MIN_32 - 1`, `INT_MAX_32 + INT_MIN_32`) to confirm the sign-comparison overflow check behaves correctly at the exact edges of the representable range.

2.9 Gray Code & Bit Reversal

What it's for: Gray code is a binary numbering system where consecutive values differ in EXACTLY one bit (unlike normal binary counting, where many bits can flip at once, e.g., 0111 → 1000). Bit reversal mirrors a number's bits end-to-end. Both are specialized bit-rearrangement techniques with recurring appearances in hardware, algorithms, and puzzles.

Use Cases:

- Gray code: rotary encoders, minimizing errors in mechanical/electronic state transitions, generating Hamiltonian paths on a hypercube, some combinatorial generation problems.
- Gray code: solving "generate all n-bit sequences such that consecutive sequences differ by one bit" style problems directly.
- Bit reversal: the core subroutine inside iterative Fast Fourier Transform (FFT) implementations (bit-reversal permutation).
- Bit reversal: reversing bits of a number for hashing, checksums, or specific bit-level puzzle problems.

Limitations:

- Gray code's "one bit differs" guarantee is between CONSECUTIVE values in the specific Gray-code ORDER — it does NOT mean any two arbitrary Gray-code values differ by one bit.
- The standard binary-to-Gray formula ($g = b \oplus (b \gg 1)$) is one-directional — converting Gray code BACK to standard binary requires a different (cumulative XOR) formula, easy to mix up.
- Bit reversal is bit-width dependent — reversing "a number" is ambiguous without specifying how many bits wide it should be treated as (8/16/32/64), since leading zeros become trailing zeros after reversal and vice versa.

Time Complexity: $O(n)$ to generate the full n-bit Gray code sequence (2^n values); $O(\log n)$ / $O(\text{bit-width})$ for a single bit reversal **Space Complexity:** $O(2^n)$ to store the full Gray code sequence; $O(1)$ for a single reversal

Approach (short): PSEUDOCODE (binary to Gray): $\text{gray} = \text{binary} \oplus (\text{binary} \gg 1)$. PSEUDOCODE (bit reversal): for each bit position i from 0 to $\text{width}-1$, take the bit at position i in the original and place it at position $(\text{width}-1-i)$ in the result.

Keywords & Constraints: "Gray code", "consecutive numbers differ by one bit", "reverse the bits", $n \leq 16-20$ for full Gray code sequence generation, fixed bit-width for reversal.

```
def binary_to_gray(n):
    """Convert a standard binary number to its Gray code equivalent."""
    return n ^ (n >> 1) # XOR each bit with the bit just above it

def gray_to_binary(gray):
    """Convert a Gray code value back to standard binary (inverse of binary_to_gray)."""
    binary = 0
    while gray:
        binary ^= gray # accumulate XOR of all remaining gray bits
        gray >>= 1
    return binary

def generate_gray_code_sequence(n):
    """Generate the full n-bit Gray code sequence: 0, 1, ..., 2^n - 1 -- consecutive values differ by 1 bit."""
    return [binary_to_gray(i) for i in range(1 <= n)]

def generate_gray_code_reflective(n):
    """Alternative construction: build recursively via the 'reflect and prefix' method."""
    if n == 0:
        return [0]
    prev = generate_gray_code_reflective(n - 1)
    # first half: prefix previous sequence with 0 (unchanged); second half: prefix REVERSED previous with 1
    return prev + [(1 <= (n - 1)) | x for x in reversed(prev)]
```

```
def reverse_bits(n, width=32):
    """Reverse the bits of an n-bit number within a given bit width."""
    result = 0
    for i in range(width):
        bit = (n >> i) & 1
        result |= (bit << (width - 1 - i))
    return result

def reverse_bits_fast(n):
    """Faster bit reversal for a fixed 32-bit width using divide-and-conquer bit swapping (no explicit loop)."""
    n = ((n & 0xaaaaaaaa >> 1) | ((n & 0x55555555 << 1) # swap odd/even bits
    n = ((n & 0xcccccccc >> 2) | ((n & 0x33333333 << 2) # swap consecutive pairs
    n = ((n & 0xf0f0f0f0 >> 4) | ((n & 0x0f0f0f0f << 4) # swap nibbles
    n = ((n & 0xff00ff00 >> 8) | ((n & 0x00ff00ff << 8) # swap bytes
    n = (n >> 16) | (n << 16) & 0xffffffff # swap 16-bit halves
    return n & 0xffffffff
```

Practice Questions — Gray Code & Bit Reversal

Q1. Generate the Gray Code Sequence Return an n-bit Gray code sequence where consecutive values differ by exactly one bit. *Approach:* Apply the formula $\text{gray} = i \oplus (i \gg 1)$ to every integer i from 0 to $2^n - 1$, which guarantees the one-bit-difference property between consecutive values.

Q2. Convert Binary to Gray Code Convert a single binary number to its Gray code representation. *Approach:* XOR the number with itself shifted right by 1 — $g = b \oplus (b \gg 1)$ directly produces the Gray code equivalent.

Q3. Convert Gray Code Back to Binary Convert a Gray code value back to its original binary representation. *Approach:* Cumulative XOR from the most significant bit downward: $\text{binary} \oplus= \text{gray}$, then shift gray right, repeating until gray is exhausted — the inverse of the forward Gray code formula.

Q4. Reverse Bits of a 32-bit Integer Reverse the bit order of a 32-bit number. *Approach:* For each bit position i in the original, place it at the mirrored position $(31-i)$ in the result, built up with repeated shifting and OR-ing.

Q5. Reverse Bits Using Divide and Conquer Reverse bits faster than a 32-iteration loop. *Approach:* Swap bits in pairs, then in groups of 2, then 4, then 8, then 16, using precomputed masks at each stage — reduces the loop to a fixed constant number of $O(1)$ operations instead of 32 iterations.

Q6. Check if a Sequence is a Valid Gray Code Sequence Given a sequence of numbers, verify that consecutive elements differ by exactly one bit. *Approach:* For each consecutive pair, XOR them and check the result has EXACTLY one set bit (using the Kernighan single-bit check from 2.1/2.2).

Q7. Circular Gray Code Property Check Verify a Gray code sequence is also circular (the LAST element differs from the FIRST by exactly one bit too). *Approach:* Same single-bit XOR check applied to the (last, first) pair in addition to all consecutive pairs — tests understanding that standard Gray code sequences are inherently circular by construction.

Q8. Total Hamming Distance in a Gray Code Sequence Sum the Hamming distances between every pair of CONSECUTIVE elements in a generated Gray code sequence. *Approach:* Since consecutive Gray code values differ by exactly 1 bit by construction, this sum trivially equals $(\text{length of sequence} - 1)$ — a good conceptual check that the Gray code property is properly understood, not something requiring computation.

Q9. Bit-Reversal Permutation (FFT Preprocessing Step) Reorder an array so element at index i moves to the position given by reversing i 's bits (fixed width based on array size). *Approach:* Compute $\text{reverse_bits}(i, \text{width})$ for every index i (where $\text{width} = \log_2(\text{array length})$), then permute the array according to those reversed indices — the standard preprocessing step used before an iterative in-place FFT.

Q10. Minimum Bit Changes to Cycle Through All N-bit Values Determine the minimum total bit flips needed to visit every n-bit value exactly once and return to start. *Approach:* A Gray code sequence achieves EXACTLY 2^n total flips (one per transition, including wraparound) since every transition (including the cyclic one) changes exactly one bit — recognizing that Gray code IS the optimal solution to this framing.

2.10 Advanced Bitmask Techniques

What it's for: A collection of more sophisticated bitmask operations beyond basic set/clear/toggle: efficiently iterating all sub-masks of a mask, combining bitmasks with dynamic programming (bitmask DP), and other compound bit tricks used in competitive-programming-level problems.

Use Cases:

- Bitmask DP — TSP, assignment problems, "which subset of items has been used" state (see DP handbook, 2.10, for the DP side; this section focuses on the bit-trick mechanics themselves).
- Enumerating all SUB-MASKS of a given mask efficiently (used inside SOS — Sum over Subsets — DP and similar advanced techniques).
- Combining multiple small bit tricks into a single compound operation (e.g., counting trailing zeros, clearing the lowest k bits at once).
- Competitive programming problems explicitly flagged with small n (≤ 20 -22) suggesting bitmask-based state compression.

Limitations:

- Naively enumerating all submasks of ALL masks (nested loop over every mask, then every submask of that mask) looks like $O(4^n)$ but is actually $O(3^n)$ total across all masks combined — a subtle but important complexity fact worth knowing rather than assuming the worst case naively.
- These techniques compound the general bitmask limitation of $n \leq 20$ -22 — SOS DP and submask enumeration don't change that fundamental ceiling, they just make what you CAN do within it more efficient.
- Advanced bit tricks (like the submask iteration trick below) have a specific, easy-to-get-wrong iteration pattern (starting the "next submask" computation from $(\text{mask} - 1) \& \text{original_mask}$) — worth verifying on a small example before trusting it in a larger solution.

Time Complexity: $O(3^n)$ total across all masks for full submask enumeration (better than the naive $O(4^n)$ estimate); $O(2^n \times n)$ or $O(2^n \times n^2)$ for typical bitmask DP **Space Complexity:** $O(2^n)$ for a DP table indexed by mask

Approach (short): PSEUDOCODE (iterate all submasks of `mask`, including mask itself and 0): `submask = mask; while submask > 0: process(submask); submask = (submask - 1) & mask;` (also separately process 0 if needed) — this visits every subset of the set bits in `mask` in $O(\text{number of submasks})$ rather than checking all 2^n possible masks and filtering.

Keywords & Constraints: "bitmask DP", "iterate all subsets of a mask", "sum over subsets", competitive programming, $n \leq 20$ -22 explicitly.

```
def iterate_all_submasks(mask):
    """Visit every submask of `mask` (every subset of its set bits), including mask itself and 0."""
    submasks = []
    submask = mask
    while submask > 0:
        submasks.append(submask)
        submask = (submask - 1) & mask
    submasks.append(0)
    return submasks
# the key trick: subtracting 1 then AND-ing with mask
# gives the NEXT smaller submask, skipping non-submasks
# 0 is technically a submask of everything (the empty subset)

def count_trailing_zeros(n):
    """Number of trailing 0 bits (position of the lowest set bit)."""
    if n == 0:
        return 32
    count = 0
    while (n & 1) == 0:
        n >>= 1
        count += 1
    return count
# convention: undefined/all-zero case
```

```

# Python alternative: (n & -n).bit_length() - 1

def clear_lowest_k_bits(n, k):
    """Clear the lowest k set bits of n (not the lowest k bit POSITIONS -- the lowest k SET bits)."""
    for _ in range(k):
        n &= (n - 1)
    return n
    # clear one lowest set bit per iteration (see 2.1)

def sum_over_subsets_dp(arr):
    """Sum-over-Subsets (SOS) DP: for every mask, sum arr[submask] over all submasks of mask, in  $O(2^n \times n)$ 
    instead of the naive  $O(3^n)$  full submask enumeration approach."""
    n = len(arr).bit_length() - 1
    dp = arr[:]
    # assumes len(arr) == 2^n
    for bit in range(n):
        for mask in range(1 << n):
            if mask & (1 << bit):
                dp[mask] += dp[mask ^ (1 << bit)]
    # pull in the contribution from the submask missing this bit
    return dp

def tsp_bitmask_dp_mechanics(dist):
    """Reference to the DP handbook's TSP bitmask DP -- shown here purely for the bit-trick mechanics involved."""
    n = len(dist)
    FULL = (1 << n) - 1
    dp = [[float('inf')] * n for _ in range(1 << n)]
    dp[1][0] = 0
    for mask in range(1 << n):
        for last in range(n):
            if not (mask & (1 << last)) or dp[mask][last] == float('inf'):
                continue
            remaining = FULL & ~mask
            # bit trick: all NOT-yet-visited cities in one step
            temp = remaining
            while temp:
                nxt_bit = temp & (-temp)
                # isolate lowest set bit = next candidate city
                nxt = nxt_bit.bit_length() - 1
                new_mask = mask | nxt_bit
                dp[new_mask][nxt] = min(dp[new_mask][nxt], dp[mask][last] + dist[last][nxt])
                temp -= nxt_bit
                # move to the next unvisited city
            return min(dp[FULL][last] + dist[last][0] for last in range(n))

```

Practice Questions — Advanced Bitmask Techniques

Q1. Iterate All Submasks of a Given Mask Enumerate every subset of the set bits in a given bitmask. *Approach:* Use the (submask - 1) & mask trick repeatedly starting from mask itself down to 0 — visits exactly the submasks of mask in $O(\text{number of submasks})$, not all 2^n possible masks.

Q2. Sum Over Subsets (SOS) DP For every mask, compute the sum of array values at all of its submasks. *Approach:* Process one bit position at a time across all masks, accumulating $dp[mask] += dp[mask \text{ without this bit}]$ — reduces what would be $O(3^n)$ naive submask enumeration to $O(2^n \times n)$.

Q3. Count Trailing Zeros Find the number of trailing zero bits in a number (equivalently, the position of its lowest set bit). *Approach:* Repeatedly right-shift while the lowest bit is 0, counting shifts; or compute $(n \& -n).bit_length() - 1$ using the lowest-set-bit isolation trick from 2.1.

Q4. Clear the Lowest K Set Bits Turn off exactly the k lowest SET bits of a number (not k bit positions). *Approach:* Apply $n \& (n-1)$ exactly k times in a loop — each application clears one more lowest set bit.

Q5. Traveling Salesman Problem — Bit-Trick Mechanics Implement the unvisited-city iteration inside a bitmask-DP TSP solution efficiently. *Approach:* Use $FULL \& \sim mask$ to get all unvisited cities as a single bitmask, then repeatedly isolate and remove the lowest set bit to iterate through candidates — avoids a full $0..n$ range scan checking membership one by one.

Q6. Maximum XOR Subset (Linear Basis / XOR Basis) Find the maximum XOR value achievable by XOR-ing any subset of an array. *Approach:* Build a linear basis of the array using Gaussian-elimination-style bit reduction (for each

number, reduce it against existing basis vectors by XOR-ing off shared high bits, inserting it as a new basis vector if it survives) — an advanced technique letting you answer "max XOR subset" in $O(n \times \text{bit-width})$ instead of $O(2^n)$.

Q7. Count Subsets with XOR Equal to a Given Value (Small N) Count how many subsets of an array XOR to exactly a target value. *Approach:* Enumerate all masks ($n \leq \sim 20$), computing the running XOR of each corresponding subset by folding over its set bits, and count matches against the target — direct brute-force bitmask enumeration application.

Q8. Minimum XOR Pair in an Array Find the pair of numbers in an array with the minimum possible XOR. *Approach:* Sort the array first — the minimum XOR pair is always ADJACENT after sorting (since XOR tends to be small between numerically close numbers sharing more high bits), reducing an apparent $O(n^2)$ pairwise problem to $O(n \log n)$.

Q9. Number of Ways to Wear Different Hats (Bitmask over People) Count assignments of n distinct hats to people, each with a preference list, one hat per person. *Approach:* DP over a bitmask representing which people already have a hat assigned; iterate hats in the outer loop, and for each hat, try assigning it to each eligible not-yet-assigned person, transitioning between bitmask states.

Q10. Shortest Superstring Problem (Bitmask DP over String Set) Find the shortest string containing every string in a given list as a substring. *Approach:* Precompute pairwise "overlap" between every pair of strings, then run a TSP-style bitmask DP ($dp[\text{mask}][\text{last}] = \text{shortest superstring using strings in } \text{mask}, \text{ ending with } \text{last}$) to find the optimal merge order — a direct application of the same bitmask-DP mechanics as TSP, applied to a string problem.

PART 3: QUICK REFERENCE CHEAT SHEET

Topic	Purpose	Time	Space	Watch Out For
Basic Bit Ops	Set/clear/toggle/check a bit	$O(1)$	$O(1)$	Off-by-one on bit index (0-indexed from LSB)
Counting Bits	Popcount / Hamming weight	$O(k)$ Kernighan	$O(1)$	Kernighan only helps when bits are sparse
XOR Properties	Find odd-one-out, swap, toggle	$O(n)$	$O(1)$	Only works when pattern is exactly even/odd occurrence
Bitmasking & Subsets	Represent/enumerate sets as ints	$O(2^n)$	$O(1)$ per mask	Only feasible for $n \leq \sim 20-25$
Shifting Tricks	Fast $\times/\div$ by 2^k , build masks	$O(1)$	$O(1)$	Forgetting the "-1" when building an n-bit mask
Arithmetic w/o Operators	Add/sub/mul via bitwise ops	$O(\log n)$	$O(1)$	Not practical, mainly educational/interview use
Single/Missing Number	Find the odd element via XOR	$O(n)$	$O(1)$	Plain XOR fails if pattern is 3x-occurrence, not 2x
Two's Complement /Overflow	Simulate fixed-width signed ints	$O(1)$	$O(1)$	Python has no native fixed width — must mask manually
Gray Code & Bit Reversal	One-bit-diff sequences, mirroring bits	$O(n)$ / $O(2^n)$	$O(1)$ / $O(2^n)$	Reversal is bit-width dependent — must specify width
Advanced Bitmask (SOS, submasks)	Efficient subset/DP mechanics	$O(3^n)$ or $O(2^n \times n)$	$O(2^n)$	Naive $O(4^n)$ assumption is wrong — true bound is $O(3^n)$

Decision Guide — “Which Pattern Do I Use?”

1. Need to read/modify one specific bit of a number? → Basic Bit Operations
2. Need to count how many bits are set? → Counting Bits & Brian Kernighan's Algorithm
3. Problem says "every element appears twice except one" or needs a temp-free swap? → XOR Properties
4. "Every element appears three times except one"? → Single Number / Missing Number Family (per-bit mod-3 counting)
5. Need to represent or enumerate all subsets of a SMALL set ($n \leq \sim 20-25$)? → Bitmasking & Subsets
6. Need fast multiply/divide by a power of 2, or to build a mask of specific width? → Shifting Tricks
7. Explicitly asked to do arithmetic WITHOUT +, -, *, or / ? → Arithmetic Without Operators
8. Problem explicitly mentions "32-bit signed integer" or overflow? → Two's Complement, Sign Bit & Overflow
9. Need consecutive values differing by exactly one bit, or to mirror a number's bits? → Gray Code & Bit Reversal

10. Small n ($\leq 20-22$) AND the state naturally is "which subset has been used/visited"? $\rightarrow$ Advanced Bitmask Techniques (pairs well with Bitmask DP in the DP handbook)